

Digital Design and Computer Organization Laboratory

UE24CS251A

3rd Semester, Academic Year 2025

Mini Project Report

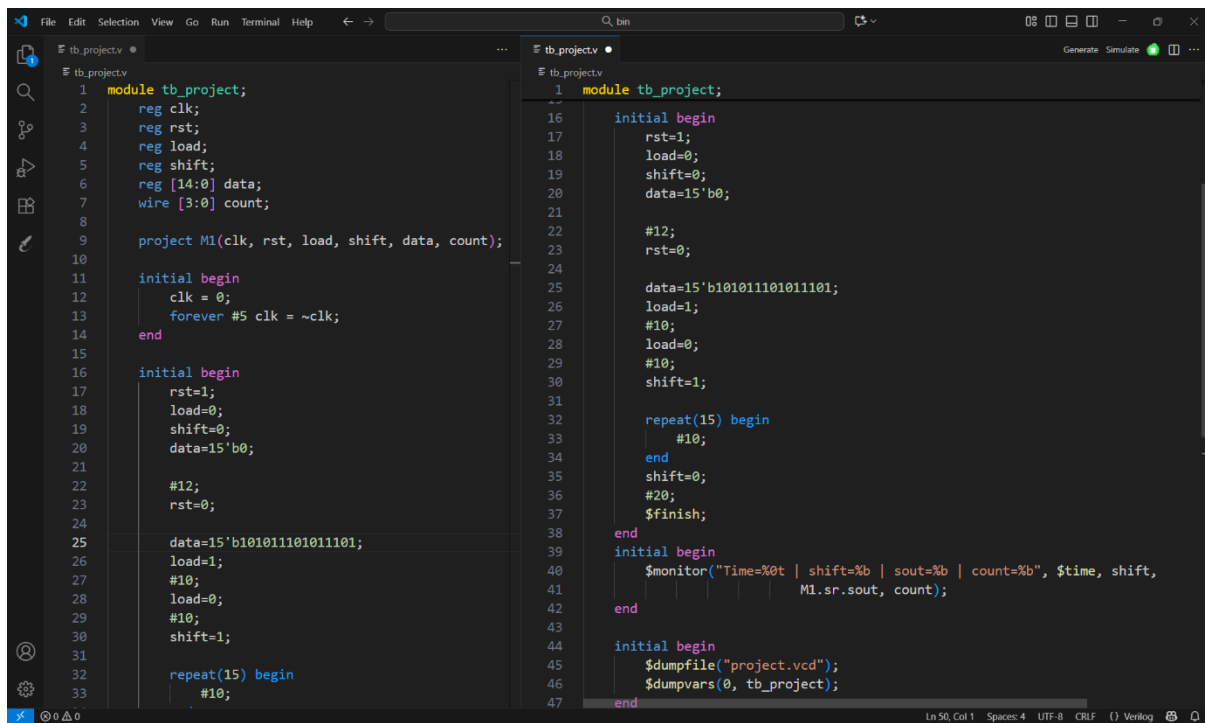
Name	SRN	Section
Dhawal Pathak	PES1UG24CS151	3 'C'
Vivian Sobers E	PES1UG24CS901	3 'C'
Aryan Upadhyay	PES1UG25CS806	3 'C'

1. Problem Statement:

Design and implement a circuit to count the number of ones in a given input data.

2. Verilog Code Screenshot

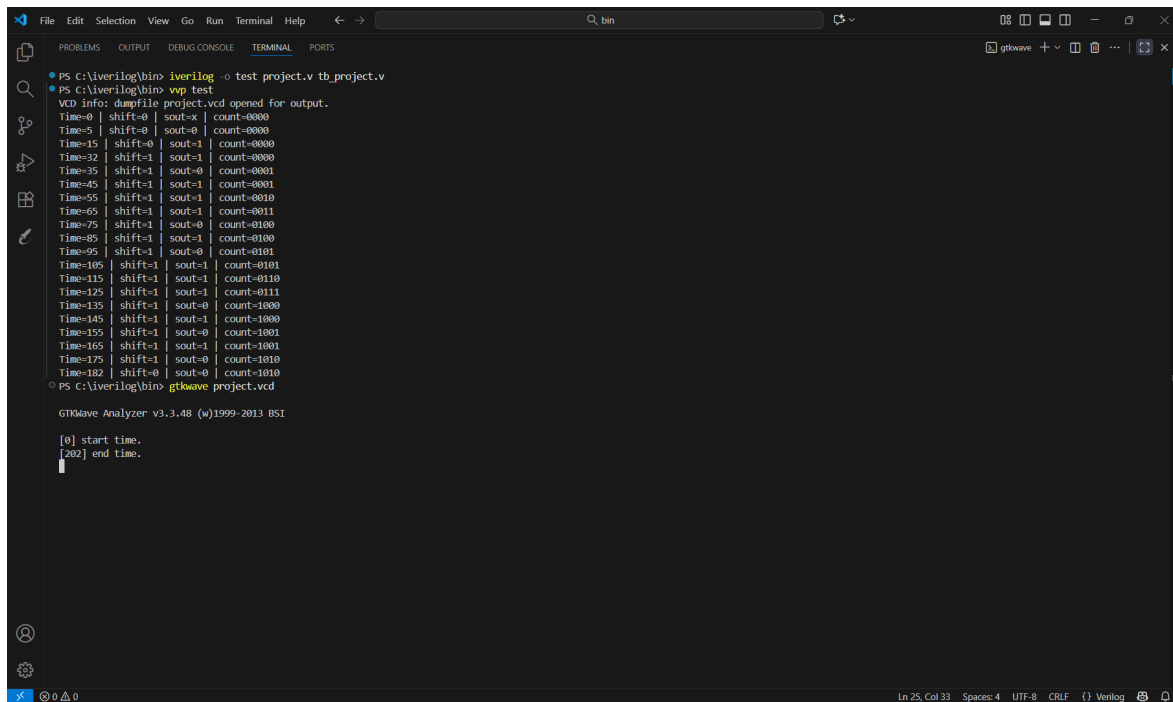
```
1 module tff(  
2     input wire clk,  
3     input wire t,  
4     input wire rst,  
5     output reg q  
6 );  
7 always @(posedge clk or posedge rst) begin  
8     if(rst)  
9         q <= 1'b0;  
10    else if(t)  
11        q <= ~q;  
12    end  
13 endmodule  
14  
15 module shift_reg(  
16     input wire clk,  
17     input wire rst,  
18     input wire load,  
19     input wire [14:0] data,  
20     input wire shift,  
21     output wire sout  
22 );  
23 reg [14:0] r;  
24 always @(posedge clk) begin  
25     if(rst)  
26         r <= 15'b0;  
27     else if(load)  
28         r <= data;  
29     else if(shift)  
30         r <= {1'b0,r[14:1]};  
31    end  
32    assign sout = r[0];  
33 endmodule  
34  
35 module counter(  
36     input wire clk,  
37     input wire rst,  
38     input wire en,  
39     output wire [3:0] q  
40 );  
41  
42 wire t0, t1, t2, t3;  
43 wire q0, q1, q2, q3;  
44  
45 assign t0 = en;  
46 assign t1 = q0 & en;  
47 assign t2 = q0 & q1 & en;  
48 assign t3 = q0 & q1 & q2 & en;  
49  
50 tff ff0(clk, t0, rst, q0);  
51 tff ff1(clk, t1, rst, q1);  
52 tff ff2(clk, t2, rst, q2);  
53 tff ff3(clk, t3, rst, q3);  
54  
55 assign q = {q3, q2, q1, q0};  
56 endmodule  
57  
58 module project(  
59     input wire clk,  
60     input wire rst,  
61     input wire load,  
62     input wire shift,  
63     input wire [14:0] data,  
64     output wire [3:0] count  
65 );  
66  
67 wire sout;  
68 wire en;  
69  
70 shift_reg sr(clk, rst, load, data, shift, sout);  
71  
72 assign en = shift & sout;  
73  
74 counter cnt(clk, rst, en, count);  
75 endmodule
```



The image shows two side-by-side windows of a text editor, both displaying Verilog code for a module named `tb_project`. The left window shows the initial setup of the module, including registers for `clk`, `rst`, `load`, `shift`, `data`, and `count`, and an initial block that sets `clk` to 0 and starts a clock generator. The right window shows the main logic of the module, including an initial block that sets `rst` to 1, `load` to 0, `shift` to 0, and `data` to `15'b0`. It also includes a `repeat(15)` block that shifts the data into the shift register and a `$monitor` statement to output the state of the shift register, `sout`, and `count`.

```
1 module tb_project;
2   reg clk;
3   reg rst;
4   reg load;
5   reg shift;
6   reg [14:0] data;
7   wire [3:0] count;
8
9   project M1(clk, rst, load, shift, data, count);
10
11   initial begin
12     clk = 0;
13     forever #5 clk = ~clk;
14   end
15
16   initial begin
17     rst=1;
18     load=0;
19     shift=0;
20     data=15'b0;
21
22     #12;
23     rst=0;
24
25     data=15'b101011101011101;
26     load=1;
27     #10;
28     load=0;
29     #10;
30     shift=1;
31
32     repeat(15) begin
33       #10;
34     end
35   end
36
37   $monitor("Time=%t | shift=%b | sout=%b | count=%b", $time, shift,
38           M1.sr.sout, count);
39
40   initial begin
41     $dumpfile("project.vcd");
42     $dumpvars(0, tb_project);
43   end
44 end
```

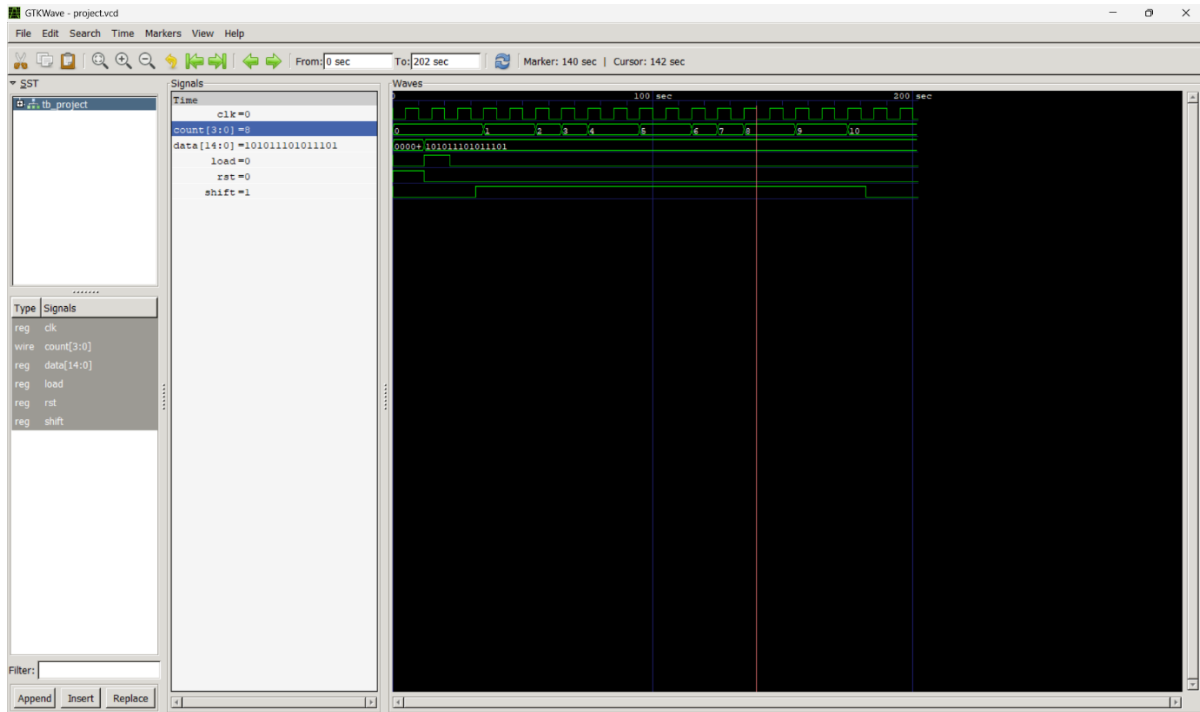
3. VVP Output Screen Shot



The image shows the output of the VVP (Verilog VPI) command in the terminal window. The output displays the state of the shift register, `sout`, and `count` at various time intervals. The output is as follows:

```
PS C:\iverilog\bin> iverilog -o test project.v tb_project.v
PS C:\iverilog\bin> vvp test
VCD info: dumpfile project.vcd opened for output.
Time=0 | shift=0 | sout=x | count=0000
Time=5 | shift=0 | sout=0 | count=0000
Time=15 | shift=0 | sout=1 | count=0000
Time=32 | shift=1 | sout=1 | count=0000
Time=35 | shift=1 | sout=0 | count=0001
Time=45 | shift=1 | sout=1 | count=0001
Time=55 | shift=1 | sout=1 | count=0010
Time=65 | shift=1 | sout=1 | count=0011
Time=75 | shift=1 | sout=0 | count=0100
Time=85 | shift=1 | sout=1 | count=0100
Time=95 | shift=1 | sout=0 | count=0101
Time=105 | shift=1 | sout=1 | count=0101
Time=115 | shift=1 | sout=1 | count=0110
Time=125 | shift=1 | sout=1 | count=0111
Time=135 | shift=1 | sout=0 | count=1000
Time=145 | shift=1 | sout=1 | count=1000
Time=155 | shift=1 | sout=0 | count=1001
Time=165 | shift=1 | sout=1 | count=1001
Time=175 | shift=1 | sout=0 | count=1010
Time=182 | shift=0 | sout=0 | count=1010
PS C:\iverilog\bin> gtwave project.vcd
GTKWave Analyzer v3.3.48 (w)1999-2013 BSI
[0] start time.
[202] end time.
```

4. GTKWAVE Screenshot



5. Logic Circuit

